10

© 한상훈 NestJS 백엔드 마스터

NestJS 백엔드 마스터

PART 1

연결 재사용, Keep-Alive, 압축

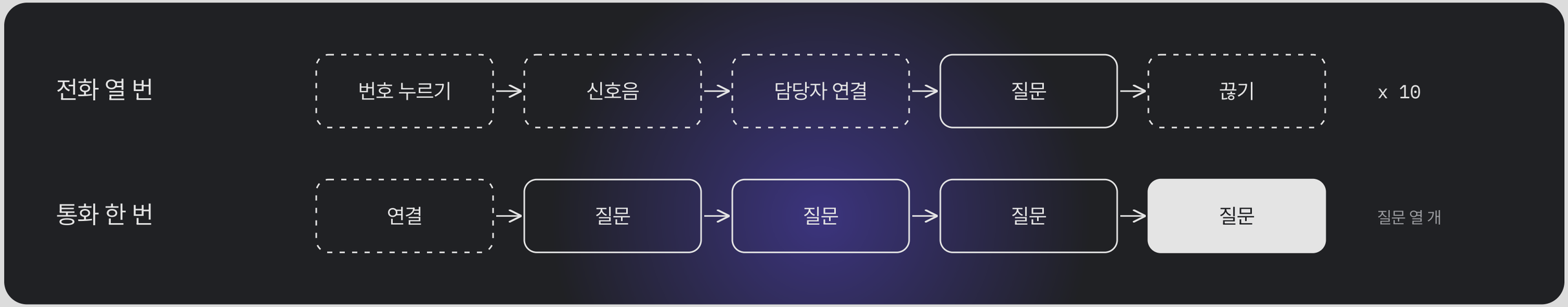
제1부 기반: 백엔드와 Node.js 런타임 · 3장 HTTP 기초

접속하는 비용과 보내는 비용 줄이기, 그 대가



© 한상훈 NestJS 백엔드 마스터

거래처 전화



→ 질문할 때마다: 번호 누르기, 신호음, 담당자 연결, 질문, 끊기

→ 통화를 열어 두면: 질문, 질문, 질문

→ 줄어드는 것: 번호 누르고 연결되기까지의 시간

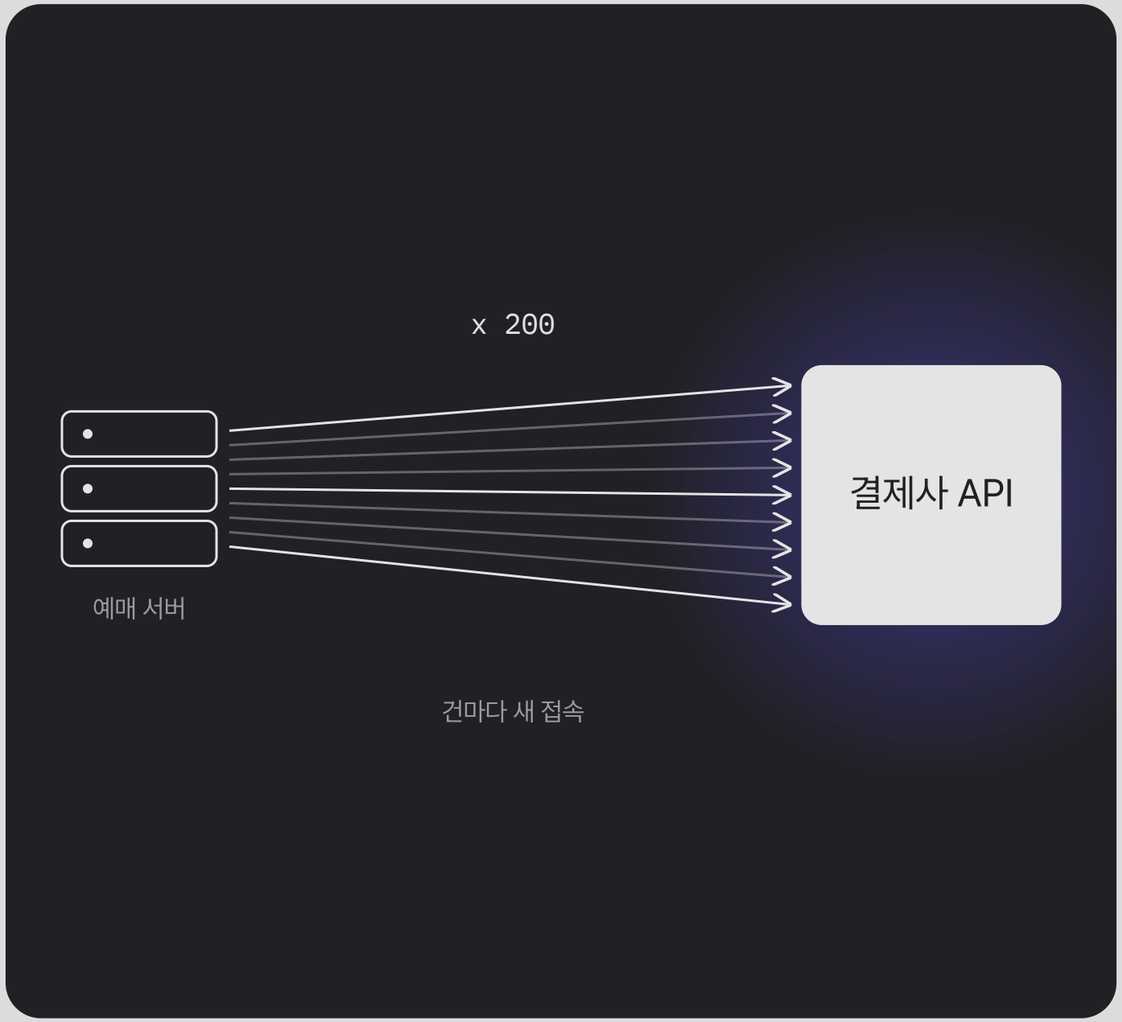
© 한상훈 NestJS 백엔드 마스터

결제사를 200번 부르는 서버

→ 예매 서버 → 결제사 API: 결제 승인, 결제 조회

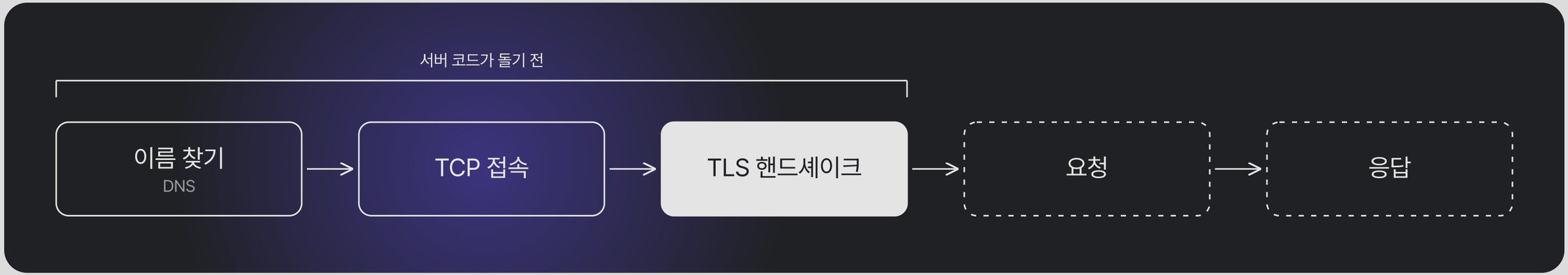
→ 오픈 직후 몇 분: 같은 결제사에 수백, 수천 건

→ 건마다 새 접속 = 건마다 전화 다시 걸기



© 한상훈 NestJS 백엔드 마스터

서버 코드에 닿기 전



→ 이름 찾기(DNS) → TCP 접속 → TLS 핸드셰이크 → 요청 → 응답

→ 앞의 세 단계: 서버 코드가 돌기 전에 쓰는 시간

© 한상훈 NestJS 백엔드 마스터

왕복 시간과 접속 비용

단계	왕복 횟수	왕복 20ms일 때
TCP 접속	1	20ms
TLS 1.3 핸드셰이크	1	20ms
요청과 응답	1	20ms + 처리 시간

→ 왕복 시간(RTT): 패킷이 상대에게 갔다가 답이 돌아오기까지

→ 새 접속: 요청을 보내기 전에 40ms

→ 20ms: 가정값

© 한상훈 NestJS 백엔드 마스터

열어 두는 연결

응답 헤더

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Connection: keep-alive
Keep-Alive: timeout=5
```

○ ○ ○ 터미널

```
$ curl -sv localhost:4010/concerts/1 localhost:4010/concerts/1 -o /dev/null -o /dev/null
* Connection #0 to host localhost:4010 left intact
* Connection #0 to host localhost:4010 left intact
```

→ HTTP/1.1: 기본으로 연결 유지, 같은 연결에 다음 요청

→ `Connection: close` → 응답 뒤 연결 종료

→ `Keep-Alive: timeout=5` → 서버가 빈 연결을 5초 동안 유지

→ 연결 하나에 요청은 한 번에 하나: 동시 요청 수만큼 연결, 쉬는 연결 재사용

→ TLS 1.2: 핸드셰이크 왕복 2회

→ 두 요청 모두 연결 #0: 첫 연결 재사용

데이터 블록: Node v24.11.0 기본값 (부록 시트) · 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

가짜 결제사

```
// fake-pg.mjs: HTTPS, 포트 5010
const seen = new WeakSet();
const server = https.createServer({ key, cert }, (req, res) => {
  if (!seen.has(req.socket)) { seen.add(req.socket); connections++; }
  res.end(JSON.stringify({ approved: true }));
});
```

→ 처음 보는 연결(소켓)로 온 요청만 세기

→ 자체 서명 인증서, 로컬 실행

© 한상훈 NestJS 백엔드 마스터

요청마다 새 Agent

```
function pay(body) {
  return new Promise((resolve, reject) => {
    const req = https.request(
      { host: 'localhost', port: 5010, path: '/pay', method: 'POST',
        agent: new https.Agent() },   // 요청마다 새로
      (res) => { res.resume(); res.on('end', resolve); },
    );
    req.on('error', reject);
    req.end(body);
  });
}
```

→ Agent: 연결을 만들고 모아 두었다가 다시 내주는 객체

→ `new https.Agent()`의 `keepAlive` 기본값: false

© 한상훈 NestJS 백엔드 마스터

200건, 연결 몇 개

부르는 방식	200건 걸린 시간	새 연결
요청마다 `new https.Agent()`	466~534ms	200개
Agent 하나 재사용 (`keepAlive: true`)	94~199ms	2개
`fetch`	235~268ms	2개

→ 같은 PC 안, 왕복 시간 거의 0

데이터 블록: 2026-10-26 강사 PC 실측, 3회씩 (부록 시트) · 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

왕복 20ms라면

	건당 접속 비용	200건 차례로
새 연결	40ms	8초 추가
연결 재사용	첫 건만	40ms 추가

→ 20ms: 가정값, 계산

→ 닫힌 연결: 한동안 TIME_WAIT 상태로 남아 포트 사용

© 한상훈 NestJS 백엔드 마스터

Agent는 한 번만

```
// 모듈을 불러올 때 한 번
const pgAgent = new https.Agent({ keepAlive: true });

https.request({ ..., agent: pgAgent }, onResponse);
```

→ Node 19부터 `https.globalAgent`의 `keepAlive` 기본값: true

→ `fetch`(undici): 기본으로 연결 재사용

→ 함정: 요청 함수 안에서 `new Agent()`, 요청마다 새 클라이언트 생성

→ 테스트에서는 정상, 결제가 몰릴 때만 느려짐

데이터 블록: Node v24.11.0 (부록 시트) · 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

서버가 연결을 닫는 시각

터미널

```
$ node ka-close.mjs
keepAliveTimeout 5000 buffer 1000
응답 헤더 Keep-Alive: timeout=5
서버가 연결을 닫음: 응답 6013ms 뒤

$ node ka-close.mjs 1000
keepAliveTimeout 1000 buffer 1000
응답 헤더 Keep-Alive: timeout=1
서버가 연결을 닫음: 응답 2008ms 뒤
```

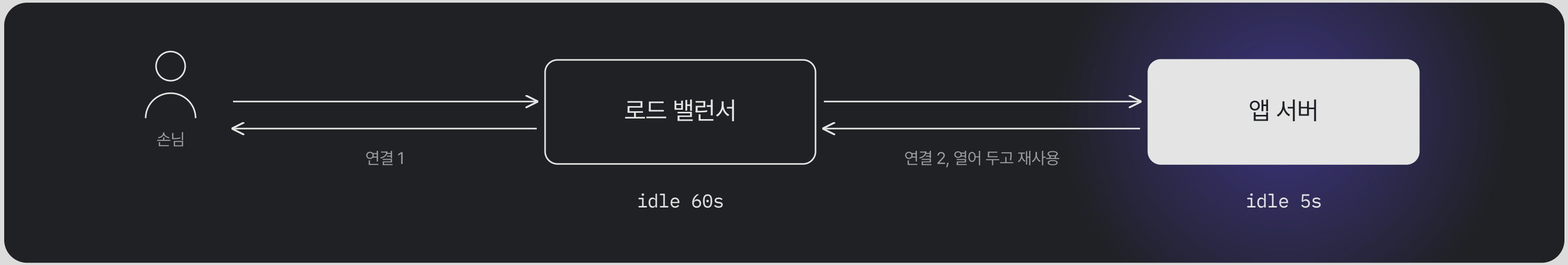
→ 헤더로 알리는 시간: `keepAliveTimeout`

→ 실제로 닫는 시각: `keepAliveTimeout` + `keepAliveTimeoutBuffer`(1초)

데이터 블록: Node v24.11.0 (부록 시트) · 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

로드 밸런서와 앱 서버의 연결



→ 손님 ↔ 로드 밸런서 ↔ 앱 서버, 연결 두 구간

→ 로드 밸런서 → 앱 서버 연결도 열어 두고 재사용

→ 유휴 시간: 로드 밸런서 60초(AWS ALB 기본값), Node 앱 서버 5초

→ 빈 연결을 먼저 닫는 쪽: 앱 서버

© 한상훈 NestJS 백엔드 마스터

닫히는 순간의 요청

두 번째 요청을 보낸 시각	20번 시도 결과
1,500ms	20번 모두 응답
1,990ms	응답 12, ECONNRESET 8
1,995ms	ECONNRESET 20
2,000ms	ECONNRESET 20
2,005ms	ECONNRESET 12, 닫힘 확인 8
2,500ms	20번 모두 닫힘 확인

→ 서버 `keepAliveTimeout` 1초(실제 닫힘 약 2초), 같은 연결로 두 번째 요청

→ ECONNRESET: 상대가 연결을 끊어 버림

→ 로드 밸런서: 이 실패를 손님에게 502로 전달

© 한상훈 NestJS 백엔드 마스터

닫는 쪽을 로드 밸런서로

```ts
// apps/ticketing/src/main.ts
const server = app.getHttpServer();
server.keepAliveTimeout = 65_000; // 로드 밸런서 유휴 시간(60초)보다 길게
```

○ ○ ○ 터미널

```
$ curl -s -D - -o /dev/null localhost:4010/concerts/1
Connection: keep-alive
Keep-Alive: timeout=65
```

→ 앱 서버 유휴 시간 > 로드 밸런서 유휴 시간

→ AWS 문서: 앱의 유휴 시간을 로드 밸런서보다 길게 설정 권장

→ nginx 업스트림 연결 재사용(keepalive)의 유휴 시간 기본값: 60초, 같은 원리

데이터 블록: ALB 유휴 시간 기본값 60초 (부록 시트) · 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

좌석 5만 석 지도

```
// GET /performances/:performanceId/seats
// 20개 구역 x 50열 x 50석 = 50,000석
{ "id": "A-3-15", "zone": "A", "row": 3, "no": 15, "status": "available" }
```

○ ○ ○ 터미널

```
$ curl -s -o /dev/null -w "%{size_download} bytes\n" localhost:4010/performances/1/seats
3189541 bytes
```

→ 좌석 지도 한 번: 약 3MB

© 한상훈 NestJS 백엔드 마스터

압축을 주고받는 헤더

```
(요청)
GET /performances/1/seats
Accept-Encoding: gzip, br

(응답)
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Vary: Accept-Encoding
Content-Encoding: br
Transfer-Encoding: chunked
```

→ `Accept-Encoding`: 클라이언트가 풀 수 있는 압축 방식

→ `Content-Encoding`: 서버가 실제로 쓴 방식

→ `Vary`: 캐시에게, 이 헤더에 따라 응답이 다름

→ 압축 뒤 `Content-Length` 없음, 조각 전송(chunked)

→ `Vary` 누락: 캐시가 br 압축본을 br 미지원 클라이언트에게 전달 → 깨진 본문

© 한상훈 NestJS 백엔드 마스터

compression 미들웨어

```
npm install compression
```

```
// apps/ticketing/src/main.ts
import compression from 'compression';
...
app.use(compression());
```

→ 미들웨어: 모든 요청이 컨트롤러 전에 거치는 함수

→ Express용 압축 미들웨어, brotli·gzip 지원

데이터 블록: compression 1.8.2 (부록 시트) · 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

압축 전후 크기

Accept-Encoding	받은 크기	10Mbps에서 전송 시간
(없음)	3,189,541 bytes	약 2.6초
gzip	295,815 bytes (9.3%)	약 0.24초
br	178,490 bytes (5.6%)	약 0.14초

→ 전송 시간: 10Mbps 가정 계산

→ 같은 PC 안 응답 시간: 압축 없음 15~22ms, gzip 37~41ms, br 22~25ms (전송이 거의 공짜, 압축 시간만 추가)

데이터 블록: 2026-10-26 강사 PC 실측 (부록 시트) · 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

압축하지 않는 작은 응답

터미널

```
$ curl -s -D - -o /dev/null -H 'Accept-Encoding: gzip, br' localhost:4010/concerts/1
HTTP/1.1 200 OK
Content-Length: 34
Vary: Accept-Encoding
```

→ 34바이트: `Content-Encoding` 없음

→ compression 기본 기준: 1KB 미만은 압축 안 함

→ 34바이트를 gzip으로: 55바이트 (오히려 커짐)

© 한상훈 NestJS 백엔드 마스터

압축에 드는 CPU 시간

방식	압축 뒤 크기	3MB 한 번 압축 시간
gzip 기본(6)	289KB	22~26ms
brotli 품질 4	174KB	13~15ms
brotli 기본 품질(11)	135KB	2,745~3,319ms

→ compression 미들웨어: brotli 품질 4로 설정 (소스 코드)

→ Node zlib의 brotli 기본 품질: 11

데이터 블록: 2026-10-26 강사 PC 실측, 5회씩 (부록 시트) · 촬영 직전 확인

© 한상훈 NestJS 백엔드 마스터

이미 압축된 데이터

```
이미 압축된 데이터(무작위 500KB) gzip: 512000 → 512178 bytes
```

→ JPEG, PNG, MP4, ZIP: 이미 압축됨, 다시 압축하면 CPU만 사용

→ compression의 기본 필터: `Content-Type`이 압축할 만한 형식일 때만

© 한상훈 NestJS 백엔드 마스터

압축을 어디서 하나

위치	장점	주의
앱 (compression)	설정 한 줄	앱 서버 CPU 사용
리버스 프록시, CDN	앱 CPU 절약, 정적 파일 미리 압축	앱 압축은 끄기

→ 큰 JSON이 자주 바뀌지 않으면: 압축본을 만들어 두고 재사용

© 한상훈 NestJS 백엔드 마스터

이 강에서 바꾼 세 줄

```ts
// apps/ticketing/src/main.ts
import { NestFactory } from '@nestjs/core';
import compression from 'compression';
import { AppModule } from './app.module.js';

async function bootstrap() {
  const app = await NestFactory.create(AppModule);
  app.use(compression());
  const server = app.getHttpServer();
  server.keepAliveTimeout = 65_000; // 로드 밸런서 유휴 시간(60초)보다 길게
  await app.listen(process.env.PORT ?? 3000);
}
await bootstrap();
```

→ 결제사 호출: Agent 하나 재사용 (결제사 연동 코드에 적용)

운영에서 볼 숫자	이상 신호
결제사로 나가는 새 연결 수	요청 수만큼 증가
로드 밸런서의 502	앱 로그에 짝 없는 502, 새벽에 몇 건씩
응답 크기, 응답 시간, CPU	큰 응답 미압축, 압축으로 CPU 과다

© 한상훈 NestJS 백엔드 마스터

열어 두는 연결, 맞추는 시간

"연결은 열어 두는 쪽이 싸고,
언제 닫을지는 양쪽이 맞춰야 합니다."